

دليل استخدام المزايا

How We Used PMS Features

شرح عملي — كيف استخدمت كل ميزة وكيف نُفِدت بشكل عام

React + Laravel + MySQL + Gemini · Multi-Tenant PMS

فهرس المحتويات

1. نموذج العمل العام (Frontend → API → Service)
2. المصادقة والجلسات
3. Multi-Tenancy — عزل الجامعات
4. موافقة الحسابات
5. إدارة المشاريع
6. الدعوات (مشرف وطالب)
7. المهام
8. التعليقات
9. الإصدارات و GitHub
10. سجل النشاط (Activity Log)
11. الإشعارات
12. لوحة التحكم
13. إدارة المستخدمين
14. الذكاء الاصطناعي (تفصيلي)
15. التوطين i18n

1. نموذج العمل العام

كل ميزة في PMS تتبع نفس النمط:

User Action (React Page) → apiFetch + Bearer Token → Laravel Route + Middleware → Controller (validation) → Service (business logic) → Model / DB / External API → JSON Response → React State Update → UI

- **Frontend:** صفحات MUI + Context (AuthContext , LanguageContext) + React.
- **Backend:** Controllers رقيقة + Services للمنطق + Eloquent Models.
- **الأمان:** Sanctum → EnsureUserHasUniversity → user.status → role middleware → ProjectAccess.

2. المصادقة والجلسات — كيف استخدمناها

من وجهة المستخدم

المستخدم يسجل أو يدخل من Login.jsx / Register.jsx . بعد النجاح يُخزّن token في localStorage ويُحمّل الملف الشخصي.

كيف نُقَدِّ

الخطوة	التفاصيل
التسجيل	AuthService::register() — ينشئ user pending · يُصدر Sanctum token · يُبلّغ admins
الدخول	AuthService::login() — يتحقق من credentials · يُرجع token + user + role
الجلسة	AuthContext يحفظ token · عند كل طلب يرسل Authorization: Bearer
تحميل الحالة	GET /profile → يحدّث status, role, university في Context
حماية الصفحات	ProtectedRoute — pending/rejected → PendingApproval · active → Dashboard
GitHub	OAuth stateless عبر token · Socialite يُحفظ في users.github_token

لماذا Sanctum؟ SPA تحتاج token — API stateless أسهل من session cookies للـ React المنفصل عن Laravel.

3. Multi-Tenancy — كيف استخدمناها

استخدمنا **Shared Database** — جامعة واحدة = tenant. كل سجل tenant-scoped له `university_id`.

الآلية

1. `BelongsToUniversity` Trait على `Models (Project, Task, ...)`.
2. `TenantScope` global scope — يفلتر تلقائياً حسب `auth()->user()->university_id`.
3. Super Admin يتجاوز `TenantScope` — يرى كل الجامعات.
4. عند الإنشاء: يُملأ `university_id` من المستخدم أو من المشروع الأب.

النتيجة: طالب جامعة A لا يرى مشاريع جامعة B — بدون تغيير في كل query يدوياً.

4. موافقة الحسابات — كيف استخدمناها

التسجيل الذاتي

1. طالب/مشرف يسجل → `status = pending`.
2. مشرف: `attachSupervisorUniversitiesPending()` — pivot pending لكل جامعة.
3. `ProtectedRoute` يمنع Dashboard — يعرض PendingApproval فقط.
4. Routes Group A (profile, logout) مسموحة حتى `pending`.

قرار المدير

→ pivot يحدّث للمشرف → `UserController::approve/reject`
: `refreshAccountStatusFromMemberships()`

- أي `membership active` → `user active`
- الكل `rejected` → `user rejected` → زر re-register
- باقي `pending` → يبقى `pending`

إنشاء من المدير

`UserController::store` — ينشئ **active** user مباشرة (بدون workflow موافقة).

5. إدارة المشاريع — كيف استخدمناها

من UI

Projects.jsx — قائمة · إنشاء · فتح · حذف.

ProjectDetails.jsx — صفحة مركزية بتبويبات (Overview, Tasks, Comments, ...).

Backend

العملية	Service / Logic
إنشاء	ProjectService — يربط user_id · university_id كمالك
الوصول	ProjectAccess::canAccess() — owner / supervisor / accepted member / admin / super_admin
التقدم	TaskService::calculateProjectProgress() — completed / total × 100
Commits	GitHub API + ProjectController::syncCommits
Activity	ProjectActivity — يُسجّل عند كل حدث مهم

List Projects → Click Project → Check ProjectAccess → Load tabs in parallel (project, tasks, progress, comments, versions)

6. الدعوات — كيف استخدمناها

دعوة مشرف

1. المالك يفتح Invites tab → يستعرض GET /supervisors .
2. يرسل invite-supervisor → سجل في supervisor_invitations .
3. المشرف يرى SupervisorInvitations.jsx → accept/reject .
4. عند القبول: projects.supervisor_id يُحدّث.

دعوة طالب

1. المالك يستعرض طلاباً متاحين (StudentService — يستبعد الأعضاء الحاليين).
2. دعوة → student_invitations · إشعار للطالب.
3. قبول → project_members.status = accepted → يحصل على ProjectAccess .

7. المهام — كيف استخدمناها

UI

`TasksTab.jsx` — عرض · إنشاء · تغيير حالة (pending/in_progress/completed) · حذف.

Backend — TaskService

1. `createTask` — ينشئ task · يسجل ProjectActivity · `notifyProjectParticipants`.
 2. `updateTask` — إذا تغيّرت الحالة → activity + notification.
 3. `deleteTask` — يتحقق owner/supervisor/member · يُعيد حساب progress.
- كل task يرث `university_id` من المشروع عبر `BelongsToUniversity` boot.

8. التعليقات — كيف استخدمناها

استخدمنا تعليقات على مستوى المشروع فقط — نموذج team chat.

التدفق

1. `CommentsTab.jsx` — يرسل نص إلى `POST /project/{id}/comment`.
2. `CommentService::addProjectComment` — يحفظ في `comments` مع `project_id`.
3. العرض: `GET /project/{id}/comments` — ترتيب زمني chat-style.
4. تعديل/حذف: `PUT/DELETE /comment/{id}` — للمالك أو صاحب التعليق.

لم نستخدم task comments في الواجهة — نقاش الفريق مركز على المشروع ككل.

9. الإصدارات و GitHub — كيف استخدمناها

Releases tab — VersionsTab.jsx

الفعل	كيف
رفع	formData → ProjectVersionService::uploadVersion → file storage/public/versions
قائمة	GET /project/{id}/versions — تُعرض في UI
تعديل	PUT title/description فقط
حذف	مالك مشروع أو مالك الإصدار · يحذف file + DB
Push GitHub	GithubService::pushVersionToGithub → github_token + مالك المشروع

GitHub OAuth

1. المستخدم يضغط Link → redirect إلى GitHub (scope: repo).
2. token → Callback stateless في users.github_token.
3. Push يرسل محتوى الملف إلى repository المشروع.

10. سجل النشاط (Activity Log) — كيف استخدمناه

تويوب **Activity log** (ProjectTimeline.jsx) — ليس timeline للإصدارات.

1. يستدعي GET /project/{id}/activities .
2. يعرض project_activities (مهام، إصدارات، انضمام، ...).
3. فلترة: Team · Releases · Tasks · All.
4. النص يُترجم عبر activityText.js + action_key.

يُكتب activity تلقائياً من Services عند: AI tasks · upload version · status change · create task · ...

11. الإشعارات — كيف استخدمناها

الإرسال

: NotificationService

- notifyUser — Laravel Database Notifications لشخص واحد.
- notifyProjectParticipants — owner + supervisor + accepted members (ما عدا الفاعل).

الاستقبال

Notifications.jsx — unread badge · paginated list · delete في sidebar · mark read ·
تُربط الإشعارات بصفحات عبر notifications.js (deep links للمشروع/المهمة).

12. لوحة التحكم — كيف استخدمناها

الدور	API	UI
الكل	/dashboard/badges · /dashboard/summary	Dashboard.jsx
Super Admin	/admin/dashboard/stats	PlatformDashboard.jsx

.Badges: unread notifications · pending invitations · pending user approvals (admin)

13. إدارة المستخدمين — كيف استخدمناها

- **Admin:** Users.jsx — CRUD · pending tab · approve/reject · password reset requests
- **Super Admin:** PlatformProjects.jsx · Universities.jsx · PlatformUsers.jsx
- فلترة: HasUniversityVisibility · inUniversity · pendingApprovalForUniversity — scopes · applyUserListFilters

14. الذكاء الاصطناعي — ملخص

للشرح التفصيلي الكامل (prompts، أخطاء، اختبارات، نقاط المناقشة): راجع 14 — PMS-AI-GUIDE.pdf فصلاً (~25 صفحة).

دمجنا Google Gemini (gemini-2.5-flash) كخدمة خارجية — مساعد للطلاب وليس autopilot.

Project Ideation 14.1

ProjectIdeation.jsx → POST /ai/suggest-projects → AIIdeationService → 3 اقتراحات · regenerate · bookmarks (max 10) · JSON مع exclude_names.

AI Task Generation 14.2

TasksTab.jsx → POST /projects/{id}/generate-tasks → AITaskService → 10 مهام · regenerate فقط إذا كل مهام AI لا تزال pending. ai_generated=true

Adopt & Generate 14.3

من بطاقة الفكرة: إنشاء مشروع + توليد مهام + انتقال لتبويب Tasks في خطوة واحدة.

14.4 التكوين والأمان

throttle: 10/h ideation · 10/min tasks · student only · sanitize · `config/ai.php` · `GEMINI_API_KEY`
+ JSON validation

15. التوطين i18n — كيف استخدمناه

- ملفات `en.js` · `ar.js` — كل النصوص الـ
- `LanguageContext` — `lang` · `dir` RTL/LTR لـ API (AI language instruction).
- `LanguageSwitcher` في Layout.
- Activity messages: `action_key` + `activityText.js` للترجمة الديناميكية.

ملخص للمناقشة

كيف نُقش استخدام المزايا:

- كل `feature = UI action + REST API + Service + Model` — نمط موحد.
- `Multi-tenant = TenantScope` تلقائي — لا `copy-paste filters`.
- `Project hub = ProjectDetails tabs` — كل `collaboration` في مكان واحد.
- `assistant = external Gemini + config prompts + strict JSON parse` — وليس `autopilot`.
- `Notifications + Activity log = شفافية للفريق` — من فعل ماذا ومتى.

PMS Features Usage Guide · `python docs/generate_pms_features_usage.py`

Companion: PMS-AI-GUIDE.pdf · PMS-PROJECT-SCOPE.pdf · PMS-ACTIVITY-DIAGRAM.pdf